SMARTERP: SECURE ROLE-BASED ENTERPRISE RESOURCE PLANNING SYSTEM FOR SMALL BUSINESSES

A PROJECT REPORT

Submitted by

SARFARAZUDDIN K (310624243166)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY,

CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"SMARTERP: SECURE ROLE-BASED ENTERPRISE RESOURCE PLANNING SYSTEM FOR SMALL BUSINESSES"** is the Bonafide work of **"SARFARAZUDDIN K (310624243166)"**, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. T. Kalai Selvi
Easwari Engineering College,
Ramapuram, Chennai – 600089.

SUPERVISOR
Ms. D. S. K. Harshitha
Easwari Engineering College,
Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate of Viva-Voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "SmartERP: Secure Role-Based Enterprise Resource Planning System for Small and Medium Enterprises" submitted by Sarfarazuddin K (310624243166) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology was conducted on 28.10.2026 in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner
Date: 28.10.2026

ABSTRACT

Small and medium enterprises (SMEs) usually run their daily operations on disconnected tools: stock is tracked in spreadsheets, orders are written on paper or in chat applications, and employee records sit in separate files. Because every record is updated by hand and at a different time, the stock shown to a salesperson often differs from what is physically available, orders are accepted for goods that no longer exist, and the owner has no reliable, current view of sales, purchases and profit. Established ERP suites solve this problem but are expensive, heavy to configure and designed for large organisations.

This project presents SmartERP, a web-based enterprise resource planning system built on the MERN stack (MongoDB, Express.js, React.js and Node.js) with a clean sky-blue interface in Tailwind CSS. The server follows a layered design in which routes, controllers, a service layer and Mongoose models are kept separate. Every operation that moves stock runs inside a MongoDB multi-document transaction, so confirming a sales order, receiving a purchase order or cancelling a confirmed order either completes fully or not at all. Stock can never become negative, a purchase order cannot be received twice, and each movement is written to a stock log. Access is governed by three roles (Admin, Manager and Staff) using one permission matrix that is shared by the Express middleware and the React navigation.

The platform provides modules for authentication and user management, inventory (categories, products, SKUs, reorder levels and low-stock alerts), sales orders with sequential invoice numbers, purchase orders with goods receipt, human resources, and a dashboard and reporting module based on MongoDB aggregation pipelines with Recharts visualisations and CSV export. The API is hardened with JWT authentication, hashed passwords, Helmet security headers, rate limiting, a CORS allow-list and request sanitisation against NoSQL injection.

In the implemented prototype an end-to-end Supertest integration flow (login, category and product creation, purchase receipt, sales confirmation and report verification) passes completely, the application is seeded with realistic demo data, and it is documented with a Postman collection and a deployment guide for MongoDB Atlas, Render and Vercel.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department Dr. T. Kalai Selvi, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide Ms. D. S. K. Harshitha, Department of Artificial Intelligence and Data Science for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	LIST OF TABLES	**iv**
	LIST OF FIGURES	**v**
1.	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	3
	1.5 MOTIVATION	4
2.	**LITERATURE SURVEY**	**5**
	2.1 LITERATURE REVIEW	5
	2.2 EXISTING SYSTEM	6
	2.3 PROPOSED SYSTEM	7
3.	**SYSTEM ANALYSIS**	**8**
	3.1 SYSTEM REQUIREMENTS	8
	3.1.1 Functional Requirements	8
	3.1.2 Non-Functional Requirements	9
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	9
	3.3 PROJECT PHASES AND ROADMAP	10
4.	**METHODOLOGY**	**11**
	4.1 SYSTEM ARCHITECTURE	11
	4.2 USER HIERARCHY AND ROLES	12
	4.3 MODULE DESCRIPTION	13
	4.4 CLASS DIAGRAM	14
	4.5 SEQUENCE DIAGRAM	15
	4.6 TRANSACTIONAL STOCK ALGORITHM	16
5.	**RESULTS AND DISCUSSION**	**18**
	5.1 IMPLEMENTATION OVERVIEW	18
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	18
	5.2.1 Permission Matrix and Authorization	18
	5.2.2 Sales Confirmation Transaction	19
	5.2.3 Purchase Receipt and Idempotency	19
	5.2.4 Cancellation and Stock Restoration	20
	5.2.5 Security Middleware Stack	20
	5.2.6 Dashboard Aggregation Pipeline	20
	5.3 RESULTS	21
	5.4 TESTING AND DISCUSSION	21
6.	**CONCLUSION & FUTURE WORK**	**23**
	6.1 CONCLUSION	23
	6.2 FUTURE ENHANCEMENTS	23
	APPENDIX: PROJECT DETAILS AND SETUP	**25**
	SCREENSHOTS OF THE PROJECT	**27**
	REFERENCES	**30**

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE No.
2.1	Comparison of SmartERP with Existing Approaches	6
3.1	Minimum Software Requirements for Development and Hosting	9
3.2	Minimum and Recommended Hardware Specifications	10
3.3	Project Phases and Completion Roadmap	10
4.1	Summary of System Modules and Core Responsibilities	13
4.2	Role Permission Matrix of SmartERP	12
4.3	Business Rules and Constraints enforced by the Transaction Engine	17
5.1	Key Results of the Implemented Prototype	21
5.2	Test Scenarios and Observed Results	21

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE No.
4.1	Layered System Architecture	12
4.2	Role-Based Access Control Hierarchy	12
4.3	UML Class Diagram of Document Models	15
4.4	Sales Order Confirmation Sequence Diagram	16
4.5	Transactional Stock Update Algorithm Flowchart	17
A.1	Login Page	27
A.2	Executive Dashboard	27
A.3	Inventory Product List	27
A.4	Sales Order and Order Confirmation	28
A.5	Purchase Order and Receive Stock	28
A.6	Reports and CSV Export	28
A.7	Employee Directory (HR Module)	29
A.8	Users and Roles Management	29

CHAPTER 1
INTRODUCTION

This chapter deals with the introduction, problem definition, objectives, scope boundaries and motivation behind the development of SmartERP, a secure role-based enterprise resource planning system for small and medium enterprises.

1.1 OVERVIEW

Every business that buys, stores and sells goods depends on three flows of information: what is in stock, what has been ordered from suppliers, and what has been sold to customers. In a small or medium enterprise (SME) these flows are often recorded in separate spreadsheets, notebooks and messaging applications. Each record is updated by hand, at a different time and by a different person, so the figures quickly drift apart. A salesperson may promise an item that was sold an hour earlier, a delivery may be received but never added to the stock sheet, and the owner can see profit only at the end of the month, if at all. Enterprise Resource Planning (ERP) software addresses this by keeping the core business processes in one integrated database.

SmartERP is a web-based ERP system designed for small and medium enterprises. It is built as a responsive MERN (MongoDB, Express.js, React.js, Node.js) application and brings inventory, sales, purchases, human resources, dashboard analytics and reporting into a single platform. Its central design idea is that stock is never changed by a plain update. Every operation that moves stock, such as confirming a sales order, receiving a purchase order or cancelling a confirmed order, is executed inside a MongoDB multi-document transaction together with its audit-log entry, so the database always remains consistent and stock can never become negative.

Beyond transactions, the system supports a complete working day of the business. Three roles, Admin, Manager and Staff, are governed by one permission matrix shared by the server and the client, so each user sees and can do only what the job requires. The dashboard presents sales trends, top-selling products and stock distribution drawn from MongoDB aggregation pipelines, reports can be filtered by date and exported as CSV, and the application is secured with JWT authentication, Helmet, rate limiting, a CORS allow-list and request sanitisation.

1.2 PROBLEM DEFINITION

Small and medium enterprises currently face the following systemic problems in managing their inventory, sales, purchases and people:

1. **Disconnected and Manual Record Keeping:** Stock, orders and employee data are kept in separate spreadsheets or on paper. Entries are duplicated, delayed and frequently inconsistent, and finding the current status of an item takes time.
2. **Inaccurate Stock and Overselling:** Without a single source of truth, orders are accepted for items that are no longer available. When two users update the same stock at the same time, one update can silently overwrite the other and the stock figure becomes wrong or even negative.
3. **No Traceability of Stock Movements:** When a quantity is wrong, there is rarely a record of who changed it, when and why. Receiving a delivery twice, or cancelling a sale without returning the goods to stock, goes unnoticed.
4. **Uncontrolled Access to Sensitive Data:** Everyone who has the spreadsheet can see purchase costs and profit margins and can edit any cell. Small businesses lack a simple way to give each staff member only the access that the job needs.
5. **Costly and Complex Enterprise Software:** Large ERP suites need long implementation, expensive licences and trained consultants, which is out of reach for most SMEs, while basic inventory apps do not cover sales, purchases, HR and reports together.

1.3 OBJECTIVE

The overall goal is to design, develop and deploy a modern, secure and reliable ERP web application that gives small and medium enterprises one consistent view of inventory, sales, purchases and people. The specific objectives are:

- **Integrated Business Modules:** Provide authentication, inventory, sales, purchases, HR, dashboard and reporting modules that share one database and one consistent data model.
- **Transaction-Safe Stock Control:** Execute every stock-changing operation inside a MongoDB transaction so that stock never becomes negative, a purchase cannot be received twice and each movement is recorded in a stock log.
- **Granular Role-Based Access Control:** Define Admin, Manager and Staff permissions in one shared permission matrix used by both the server middleware and the client navigation.

- **Actionable Analytics:** Present KPIs, sales trends, top products and stock distribution using aggregation pipelines and Recharts, with date-filtered reports and CSV export.
- **Defence in Depth Security:** Protect the API with JWT authentication, hashed passwords, Helmet security headers, rate limiting, a strict CORS allow-list and NoSQL-injection sanitisation.
- **Quality Assurance:** Verify the system with unit, service and Supertest end-to-end integration tests that cover the complete login-to-report workflow.
- **Modern and Accessible User Experience:** Deliver a responsive sky-blue interface with skeleton loaders, empty states, toast feedback, confirmation dialogs and visible keyboard focus styles.
- **Deployment Readiness:** Provide demo seed data, API documentation as a Postman collection and a deployment guide for MongoDB Atlas, Render and Vercel.

1.4 SCOPE OF THE PROJECT

A. In-Scope Deliverables

1. **Authentication and User Management:** JWT login, role assignment and Admin-only user and role management, with protection against Admin self-lockout.
2. **Inventory Management:** Categories, products with unique SKUs, prices, stock quantities and reorder levels, low-stock alerts and a stock-movement log.
3. **Sales Management:** Customer records, sales orders with line items and totals, order confirmation that reduces stock, sequential invoice numbers and cancellation with stock restoration.
4. **Purchase Management:** Supplier records, purchase orders and a goods-receipt operation that increases stock exactly once.
5. **HR Management:** An employee directory linked to user accounts, visible to the authorised roles.
6. **Dashboard and Reports:** KPI cards, sales trend, top products and category stock charts, date-filtered sales and purchase reports with CSV export, and a limited dashboard for Staff with cost and profit hidden.
7. **Quality, Security and Delivery:** Automated tests, security middleware, a demo seed script, a Postman collection, a README and a deployment guide.

B. Out-of-Scope Boundaries

- Payment gateway integration and online payment processing.
- Accounting modules such as general ledger, tax filing and payroll processing.
- Multi-warehouse, multi-company (multi-tenant) operation and barcode-scanner hardware integration.
- Native mobile applications; the interface is a responsive web application only.

1.5 MOTIVATION

Stock and cash are the two most valuable resources of a small business, yet they are usually managed with the least reliable tools. The owner or a trusted employee spends hours every week reconciling sheets, correcting mistakes and answering the question of what is really in stock. Errors are expensive: an oversold item damages customer trust, and an unnoticed shortage stops sales. At the same time, modern web technologies and well-understood database guarantees, namely ACID transactions and role-based access control, make it possible to build a reliable system with an open-source stack. The motivation of this project is to give small businesses a tool that keeps stock correct under every operation, shows the state of the business clearly to each role, and can be deployed in the cloud within an hour.

CHAPTER 2
LITERATURE SURVEY

This chapter deals with a survey of the concepts behind enterprise resource planning, transaction processing and access control, a critical look at the existing approaches, and the proposed SmartERP system.

2.1 LITERATURE REVIEW

Computer-based management of business resources began with Material Requirements Planning (MRP) in the 1960s and 1970s, which was extended to Manufacturing Resource Planning (MRP II) and, in the 1990s, to Enterprise Resource Planning. Davenport (1998) described how an ERP system replaces fragmented information systems with a single integrated database, and Monk and Wagner (2013) show that the main benefit for an organisation is consistent, real-time information shared across departments. The same studies note that the cost and complexity of enterprise-class systems have kept many small and medium enterprises away from them.

Reliable stock control is fundamentally a transaction-processing problem. The ACID properties of atomicity, consistency, isolation and durability were formalised by Härder and Reuter (1983) and described in depth by Gray and Reuter (1993). An order that decreases the stock of several products must either complete for all of them or for none, and two concurrent orders must not read the same stale quantity. Relational databases have offered such guarantees for decades; document databases such as MongoDB added multi-document ACID transactions on replica sets in version 4.0, which allows the flexibility of the document model to be combined with protected multi-step business operations. Kleppmann (2017) explains why such guarantees matter and how weaker isolation leads to anomalies such as lost updates.

Access control is the second pillar. Ferraiolo and Kuhn (1992) and Sandhu et al. (1996) introduced Role-Based Access Control (RBAC), in which permissions are attached to roles and users receive roles instead of being granted rights one by one. RBAC mirrors the structure of a business, reduces administrative effort and supports the principle of least privilege. The OWASP Top Ten (2021) ranks broken access control as the most serious web application risk and lists injection, including NoSQL injection, among the common attacks, which motivates server-side enforcement of permissions and sanitisation of all request data.

On the engineering side, token-based authentication with JSON Web Tokens (Jones, Bradley and Sakimura, 2015) allows a stateless API to be deployed independently of the client, and a layered architecture that separates controllers, services and data models (Sommerville, 2015) improves maintainability and testability. Together these studies suggest a practical design for an SME-oriented ERP: one integrated data model, transaction-protected stock operations, a centrally defined RBAC matrix and a layered, well-tested web architecture. These are the foundations of SmartERP.

2.2 EXISTING SYSTEM

The approaches commonly used by small and medium enterprises can be grouped into three categories:

1. **Manual and Spreadsheet-Based Management:** Stock, sales and purchase records are maintained on paper or in spreadsheets. There is no automatic stock update, no protection against overselling, no audit trail and no access control, and reports have to be compiled by hand.
2. **Basic Inventory and Billing Applications:** Simple desktop or mobile tools track stock and print bills but usually do not cover purchases, HR and analytics in one place, offer limited role control and give little protection against concurrent updates.
3. **Enterprise and Open-Source ERP Suites:** Products such as SAP, Oracle, Odoo and ERPNext are powerful and feature-rich, but they require long configuration and training and often paid licences or consultants. Their breadth makes the interface and permission model complex for a small team that needs only the core modules.

Table 2.1: Comparison of SmartERP with Existing Approaches

Feature	Existing Approaches (Spreadsheets / Enterprise ERP)	SmartERP (Modern MERN)
UI and Usability	Plain sheets or dense, module-heavy enterprise screens	Clean sky-blue responsive interface with cards, charts, skeleton loaders and toast feedback
Stock Consistency	Manual updates; overselling and negative stock are possible	Transactional updates; insufficient stock is rejected and stock never becomes negative
Audit Trail	Rarely available or limited	Every stock movement is logged with its reference and timestamp
Access Control	Shared files or complex configuration	Three roles driven by one shared permission matrix; costs hidden from Staff

Feature	Existing Approaches (Spreadsheets / Enterprise ERP)	SmartERP (Modern MERN)
Reporting	Manual compilation	Aggregation-based dashboard, date-filtered reports and CSV export
Setup and Deployment	Local installation or lengthy implementation	Cloud-ready on Atlas, Render and Vercel with demo seed data
Cost	Low for sheets, high for licensed suites	Open-source stack with no licence fee

2.3 PROPOSED SYSTEM

The proposed system, SmartERP, is a modern web application that brings together transaction-safe stock control, role-based access, analytics and a secure API in one place. It introduces four main ideas:

1. **Transactional Stock Engine:** Sales confirmation, purchase receipt and order cancellation run inside MongoDB transactions with conditional atomic updates, so concurrent operations cannot oversell stock and a failure in the middle leaves no partial data.
2. **Single-Source Role-Based Access Control:** One permission matrix is read by the Express authorisation middleware and by the React navigation, so the screens a user sees and the actions the server allows can never disagree.
3. **Layered Service Architecture:** Controllers stay thin and the business rules live in a service layer that is covered by unit and integration tests, which keeps every module consistent and maintainable.
4. **Secure, Tested and Deployable Platform:** JWT authentication, Helmet, rate limiting, a CORS allow-list and sanitisation protect the API, an end-to-end test proves the complete workflow, and the system is documented for cloud deployment.

CHAPTER 3
SYSTEM ANALYSIS

This chapter deals with the functional and non-functional requirements of the system, the software and hardware environment, and the project phases.

3.1 SYSTEM REQUIREMENTS

3.1.1 Functional Requirements

- **FR-1 Authentication:** Users log in with an email and password and receive a JWT. Three roles are supported: Admin, Manager and Staff.
- **FR-2 User and Role Management:** The Admin creates users, assigns roles and changes them, and cannot delete or demote the last Admin account (self-lockout protection).
- **FR-3 Inventory Management:** Managers and Admins maintain categories and products with a unique SKU, price, cost, stock quantity and reorder level; products at or below the reorder level are flagged as low stock.
- **FR-4 Purchase Management:** Suppliers and purchase orders can be created, and receiving a purchase order increases stock exactly once and writes a stock-log entry.
- **FR-5 Sales Management:** Customers and sales orders can be created; confirming an order reduces stock, generates a sequential invoice number and writes a stock-log entry, and an order with insufficient stock is rejected.
- **FR-6 Cancellation and Rollback:** Cancelling a confirmed sales order restores the stock and records the reverse movement in the stock log.
- **FR-7 HR Management:** An employee directory is linked to user accounts and is visible to the roles that are permitted to view it.
- **FR-8 Dashboard:** KPI cards and charts show sales trend, top products and category stock distribution; the Staff dashboard hides cost and profit values.
- **FR-9 Reports and Export:** Sales and purchase reports can be filtered by date range and exported as a CSV file.
- **FR-10 User Feedback:** The interface shows skeleton loaders, empty states, toast notifications and confirmation dialogs before irreversible actions.
- **FR-11 Demo Data:** A seed script fills the database with demo users, categories, products, partners, orders and employees.

3.1.2 Non-Functional Requirements

- **NFR-1 Data Integrity:** Stock quantities must never become negative, and every stock-changing operation must be atomic.
- **NFR-2 Security:** Every API route is protected by authentication and role checks; passwords are stored in hashed form and never returned by the API; requests are rate limited, sanitised and restricted by a CORS allow-list.
- **NFR-3 Performance:** Read operations and dashboard queries should feel instantaneous, using MongoDB indexes and aggregation pipelines.
- **NFR-4 Usability and Accessibility:** A responsive sky-blue design with 12 px rounded cards, clear typography, labelled inputs and visible keyboard focus styles that works on mobile, tablet and desktop.
- **NFR-5 Maintainability:** A clear separation of controllers, services and models and a shared permission file so that rules are defined once.
- **NFR-6 Testability:** Services are covered by unit tests and the complete workflow by a Supertest integration test.
- **NFR-7 Portability:** The React client and the Express server can be deployed and scaled independently on the cloud, with all settings supplied through environment variables.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The software and hardware environments used for development and hosting are listed in Tables 3.1 and 3.2.

Table 3.1: Minimum Software Requirements for Development and Hosting

Software Layer	Component	Version	Role in SmartERP
Operating System	Windows 10/11, Linux or macOS	64-bit	Development and hosting
Runtime	Node.js	Current LTS	Runs the Express API and service layer
Backend Framework	Express.js	4.x	REST API and middleware
Database	MongoDB Atlas (replica set)	6.x or later	Document storage; the replica set enables transactions
ODM	Mongoose	Latest stable	Schemas, models and transaction sessions
Frontend Library	React.js with Vite	Latest stable	Single-page application
Styling	Tailwind CSS	3.x	Sky-blue design system

Software Layer	Component	Version	Role in SmartERP
Charts	Recharts	Latest stable	Dashboard visualisations
Testing	Jest and Supertest	Latest stable	Unit, service and integration tests
API Testing	Postman	Latest	API documentation collection
Package Manager	npm	Bundled with Node.js	Installing and running client and server
Browser	Chrome, Edge or Firefox	Latest	Accessing the web application

Table 3.2: Minimum and Recommended Hardware Specifications

Hardware Component	Minimum	Recommended
Processor	Intel Core i3 / Ryzen 3 (2.5 GHz)	Intel Core i5 / Ryzen 5 or better
Memory (RAM)	4 GB	8 GB or more
Storage	5 GB free space	10 GB SSD
Network	Broadband connection	Stable broadband for cloud hosting
Display	1366 x 768	1920 x 1080

3.3 PROJECT PHASES AND ROADMAP

The project followed a structured six-phase development roadmap over eight weeks, summarized in Table 3.3. The first five phases are complete; testing, deployment and the final documentation constitute the last phase.

Table 3.3: Project Phases and Completion Roadmap

Phase and Milestone Description	Timeline	Status
Phase 1: Requirement Analysis, Literature Survey, Architecture and Project Setup	Week 1	Completed
Phase 2: Backend Foundation: Authentication, Users and Role-Based Access Control	Week 2 - 3	Completed
Phase 3: Inventory, Sales and Purchases with Transactions and Stock Logs	Week 4 - 5	Completed
Phase 4: HR Module, Permission Matrix and Frontend Integration	Week 6	Completed
Phase 5: Dashboard, Aggregation Reports, Recharts Visualisations and CSV Export	Week 7	Completed
Phase 6: Testing, Security Hardening, Seed Data, Cloud Deployment and Final Documentation	Week 8	In progress

CHAPTER 4
METHODOLOGY

This chapter deals with the design of SmartERP: the layered architecture, user hierarchy, module description, class diagram, sequence diagram and the transactional stock algorithm.

4.1 SYSTEM ARCHITECTURE

SmartERP follows a layered architecture in a single repository (monorepo) that separates presentation, API, business services, data access and persistence, as shown in Figure 4.1.

1. **Presentation Layer:** A React.js single-page application built with Vite and styled with Tailwind CSS provides the pages, the role-aware sidebar and the Recharts dashboards. It runs as the front-end web app on port 5173 during development.
2. **API and Security Layer:** An Express.js REST API (port 5000) applies Helmet headers, the CORS allow-list, rate limiting and request sanitisation, authenticates the JWT, checks the role permission and validates input before the request reaches a controller.
3. **Service Layer:** The core of the system. The inventory, sales, purchase and report services contain all business rules and open the MongoDB transactions that change stock, so controllers stay thin and the rules can be tested without HTTP.
4. **Data Access Layer:** Mongoose models for users, employees, categories, products, customers, suppliers, sales orders, purchase orders and stock logs isolate the rest of the code from database details.
5. **Persistence Layer:** MongoDB Atlas, running as a replica set, stores all documents. Indexes on the SKU and on order dates support fast look-ups, and a seed script provides demo data.

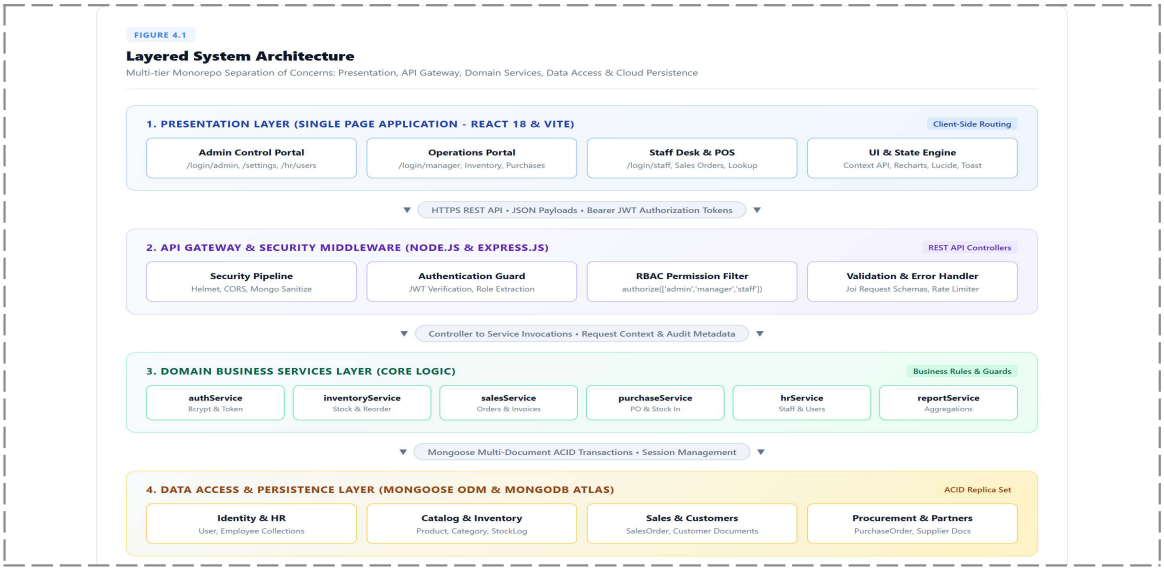


Figure 4.1: Layered System Architecture

4.2 USER HIERARCHY AND ROLES

The role hierarchy of Figure 4.2 is reflected directly in the permissions of the application. The Admin is the master administrator who manages users and roles, system settings and all modules. The Manager runs the daily business: inventory, purchases, sales and reports, and can view employees but cannot manage users. The Staff role is intended for counter and sales work: it can view inventory, create sales orders and see a limited dashboard from which cost and profit values are removed. The permissions of the three roles are summarised in Table 4.2.



Figure 4.2: Role-Based Access Control Hierarchy

Table 4.2: Role Permission Matrix of SmartERP

Capability	Admin	Manager	Staff
View dashboard	Full	Full	Limited (cost and profit hidden)
View inventory and stock logs	Yes	Yes	View only

Capability	Admin	Manager	Staff
Create, edit and delete products and categories	Yes	Yes	No
Create sales orders	Yes	Yes	Yes
Confirm and cancel sales orders	Yes	Yes	No
Create and receive purchase orders	Yes	Yes	No
View reports and export CSV	Yes	Yes	No
View employees (HR)	Yes	Yes	No
Manage users, roles and system settings	Yes	No	No

4.3 MODULE DESCRIPTION

The application consists of seven cooperating modules:

1. **Authentication and User Management:** Handles login, JWT issue and verification, role assignment and Admin-only user management, and prevents the Admin from locking themselves out.
2. **Inventory Management:** Maintains categories and products with unique SKUs, prices, stock and reorder levels, raises low-stock alerts and keeps the stock-movement log.
3. **Sales Management:** Manages customers and sales orders, validates stock, confirms orders inside a transaction, generates sequential invoice numbers and supports cancellation with stock restoration.
4. **Purchase Management:** Manages suppliers and purchase orders and receives goods inside a transaction so that stock increases exactly once.
5. **HR Management:** Stores the employee directory linked to user accounts and shows it to the permitted roles.
6. **Dashboard:** Shows StatCards, the sales trend line chart, the top products bar chart and the category stock pie chart, using aggregation pipelines and role-based redaction of cost and profit.
7. **Reports and Export:** Provides date-filtered sales and purchase reports in tabular form and streams them as CSV downloads.

Table 4.1: Summary of System Modules and Core Responsibilities

Code	Module	Primary Entities	Key Responsibility
MOD-01	Authentication and Users	User, Role	JWT login, RBAC, user management

Code	Module	Primary Entities	Key Responsibility
MOD-02	Inventory	Category, Product, StockLog	SKU catalogue, reorder alerts, stock history
MOD-03	Sales	Customer, SalesOrder	Confirm order, invoice number, cancel and restore
MOD-04	Purchases	Supplier, PurchaseOrder	Create order and receive stock once
MOD-05	HR	Employee, User	Employee directory linked to users
MOD-06	Dashboard	Aggregations, KPIs	Charts and KPI cards with role-based redaction
MOD-07	Reports and Export	SalesOrder, PurchaseOrder	Date-filtered reports and CSV streaming

4.4 CLASS DIAGRAM

The data model is a set of MongoDB document collections. Figure 4.3 shows the main entities. A User account holds the email, hashed password and role, and may be linked to an Employee profile. A Category groups many Products, and each Product stores a unique SKU, price, cost, stock quantity and reorder level. A SalesOrder references one Customer and contains line items that reference Products, together with totals, a status and the invoice number. A PurchaseOrder references one Supplier and its line items in the same way. Every change of stock creates a StockLog document that records the Product, the quantity change, the reference type and the reference document.

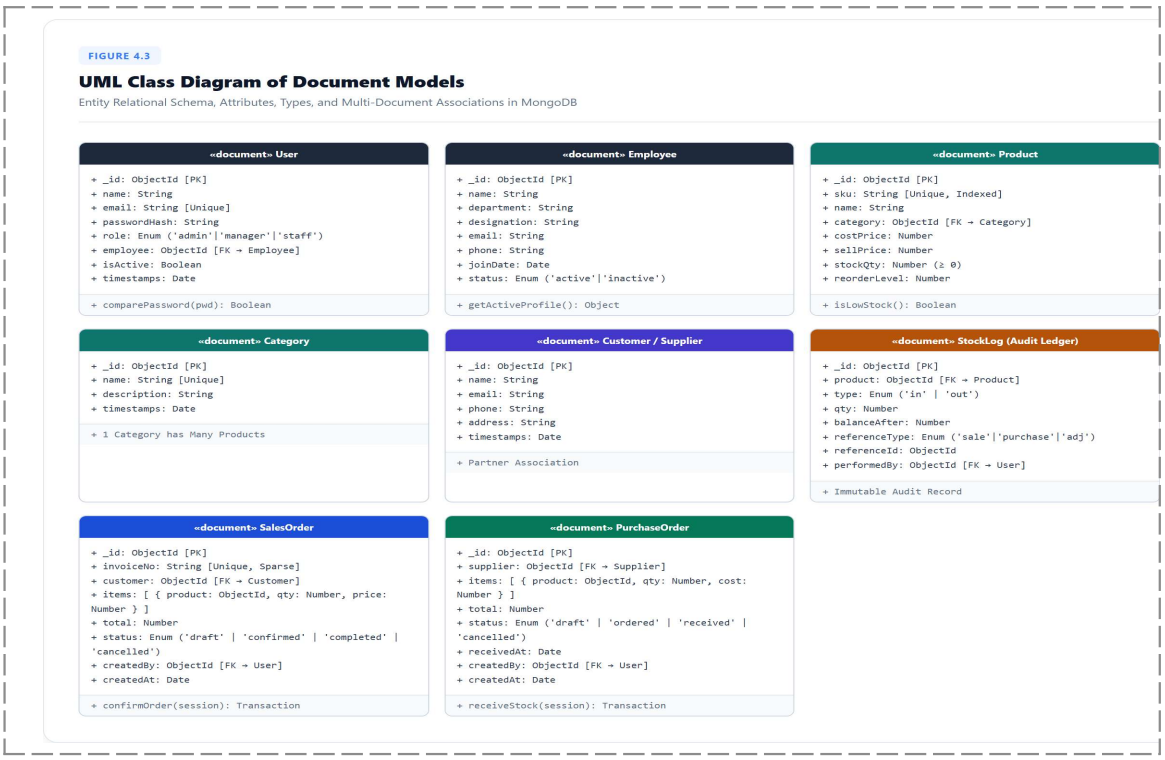


Figure 4.3: UML Class Diagram of Document Models

4.5 SEQUENCE DIAGRAM

Figure 4.4 shows what happens when a user confirms a sales order. The React client sends the request with the JWT to the Express API, which applies the security middleware, verifies the token and checks that the role is allowed to confirm orders. The controller calls the sales service, which opens a database session and starts a transaction, reads the order, decrements the stock of each line item with a conditional atomic update, writes the stock-log entries, assigns the next invoice number and sets the status to confirmed. When everything succeeds the transaction is committed and the confirmed order is returned, and the interface shows a success toast and refreshes the list. If any step fails, the transaction is aborted, no data is changed and an error message is returned.

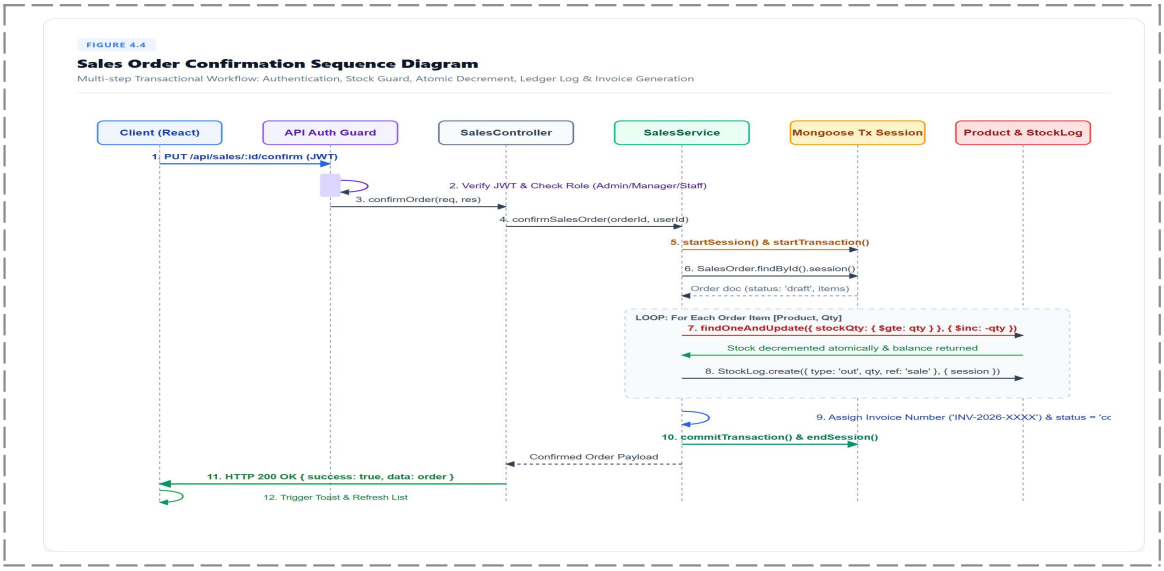


Figure 4.4: Sales Order Confirmation Sequence Diagram

4.6 TRANSACTIONAL STOCK ALGORITHM

Every operation that changes stock has the same shape, summarized in Figure 4.5. The request is first authenticated, authorised, validated and sanitised. The service then starts a session and a transaction. Inside the transaction it reads the document, checks the business guard (order status, available stock), applies the change with an atomic conditional update, writes the stock-log entry and the new status, and commits. If any step throws an error the whole transaction is aborted, nothing is saved and the controller returns a clear message.

The invariant protected by the algorithm is that the stock of every product equals its opening stock plus everything received and restored, minus everything sold, and is never below zero:

$$\text{Stock}(p) = \text{Opening}(p) + \text{Received}(p) + \text{Restored}(p) - \text{Sold}(p) \geq 0$$

where Received(p) is the sum of quantities of received purchase orders, Sold(p) the sum of quantities of confirmed sales orders, and Restored(p) the sum of quantities returned by cancelled confirmed orders. The sale of a line item is applied only if the stock is at least the ordered quantity, so the invariant holds even when two users confirm orders for the same product at the same time. The business rules enforced by the engine are listed in Table 4.3.

Table 4.3: Business Rules and Constraints enforced by the Transaction Engine

Type	Rule	Handling
Hard	Stock of a product can never become negative	Conditional atomic update; the whole sale is aborted with an insufficient stock error
Hard	A purchase order can be received only once	Status is checked inside the transaction (idempotency guard)
Hard	Confirming a sales order reduces stock and writes a stock log	Same transaction; commit or roll back together
Hard	Cancelling a confirmed order restores stock with an audit log	Reverse movement inside a transaction
Hard	Invoice numbers are sequential and unique	Generated inside the transaction from a sequence counter
Hard	Every route needs a valid JWT and an allowed role	Authentication and authorizeAction middleware
Hard	The Admin cannot delete or demote their own account	Self-lockout guard in the user service
Soft	Products at or below the reorder level are flagged	Low-stock alert on dashboard and inventory
Soft	Staff must not see cost and profit values	Redaction of the dashboard response by role

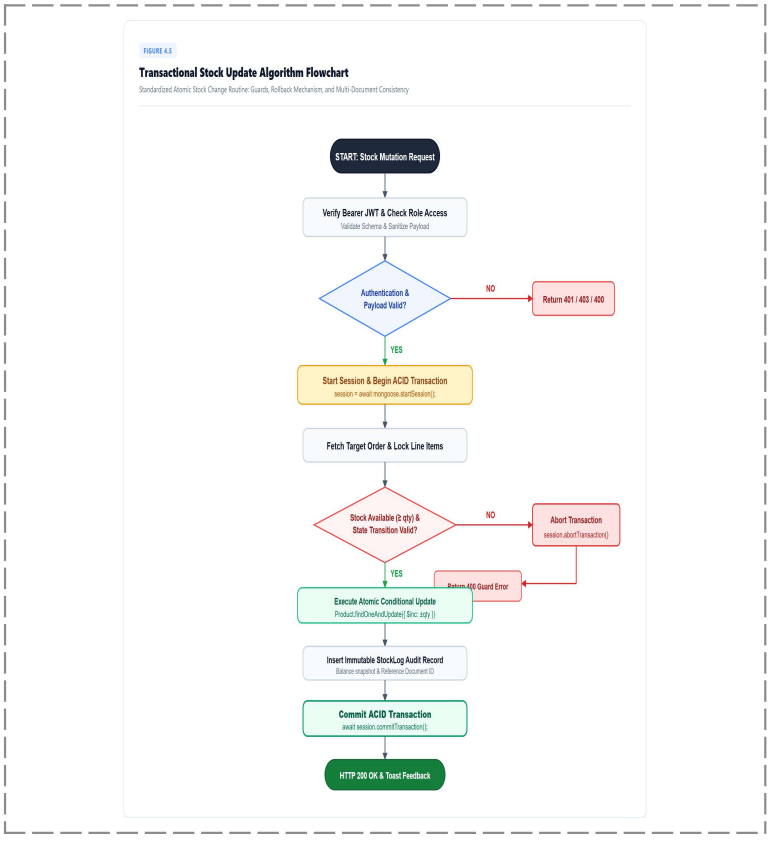


Figure 4.5: Transactional Stock Update Algorithm Flowchart

CHAPTER 5
RESULTS AND DISCUSSION

This chapter deals with the implementation of SmartERP, the algorithmic walkthrough of its key functions, and the results obtained from the prototype.

5.1 IMPLEMENTATION OVERVIEW

The system is implemented as a monorepo with a server (Node.js with Express.js, started with npm run dev, serving the REST API and the service layer on port 5000), a client (React.js with Vite and Tailwind CSS, started with npm run dev, serving the web application on port 5173) and a shared folder that holds the permission matrix. MongoDB Atlas holds the data, and the seed script fills it with three users (one per role), five categories, thirty products with realistic SKUs and prices, customers, suppliers, twenty historical orders and employees. Because transactions need a replica set, the system is run against an Atlas cluster, which is a replica set by default.

On the client side the interface is organized as pages for the Dashboard, Inventory, Sales, Purchases, HR, Reports and Users, shown through a collapsible sidebar that is built from the shared permission matrix. Skeleton loaders, empty-state components, toast notifications and a reusable confirmation dialog give consistent feedback. On the server side the API enforces the role rules, and every operation that changes stock, whether it is a sale, a purchase receipt or a cancellation, runs through the same transactional service layer, which guarantees that stock can never be left inconsistent.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The following listings show the core algorithms in simplified JavaScript. They illustrate the logic of the implementation; identifiers can differ slightly from the project source.

5.2.1 Permission Matrix and Authorization

The permission matrix is a single JSON file shared by the server and the client. The server middleware looks up the allowed roles of an action and rejects the request with status 403 when the role of the authenticated user is not listed. The client reads the same file to hide menu items and routes.

```
// shared/permissions.json (excerpt)
{ "sales.create": ["admin", "manager", "staff"],
  "purchases.receive": ["admin", "manager"],
  "users.manage": ["admin"] }
```

```
// server/middleware/authorizeAction.js
const permissions = require('../../shared/permissions.json');
function authorizeAction(action) {
  return (req, res, next) => {
    const allowed = permissions[action] || [];
    if (!allowed.includes(req.user.role)) return res.status(403).json({ message: 'Forbidden' });
    next();
  };
}
```

5.2.2 Sales Confirmation Transaction

Confirming a sales order opens a transaction. For each line item the product is updated only if its stock is at least the ordered quantity, which makes the check and the decrement a single atomic step. If any line fails, the error aborts the transaction and no stock is changed. On success the stock logs, the invoice number and the new status are saved together.

```
// server/services/salesService.js
async function confirmOrder(orderId, userId) {
  const session = await mongoose.startSession();
  try {
    let result;
    await session.withTransaction(async () => {
      const order = await SalesOrder.findById(orderId).session(session);
      if (order.status !== 'draft') throw new AppError('Order already processed', 409);
      for (const line of order.items) {
        const updated = await Product.findOneAndUpdate(
          { _id: line.product, stock: { $gte: line.quantity } },
          { $inc: { stock: -line.quantity } }, { session, new: true });
        if (!updated) throw new AppError('Insufficient stock', 400);
        await StockLog.create([{ product: line.product, change: -line.quantity,
          refType: 'sales', refId: order._id, user: userId }], { session });
      }
      order.invoiceNumber = await nextInvoiceNumber(session);
      order.status = 'confirmed';
      result = await order.save({ session });
    });
    return result;
  } finally { session.endSession(); }
}
```

5.2.3 Purchase Receipt and Idempotency

Receiving a purchase order increases the stock of each product and writes the stock logs. The status check inside the transaction makes the operation idempotent: a second attempt to receive the same order, for example by a double click or a repeated request, is rejected and cannot add the quantity twice.

```
// server/services/purchaseService.js
async function receivePurchase(poId, userId) {
  const session = await mongoose.startSession();
  try {
    await session.withTransaction(async () => {
      const po = await PurchaseOrder.findById(poId).session(session);
      if (po.status === 'received') throw new AppError('Already received', 409);
      for (const line of po.items) {
        await Product.updateOne({ _id: line.product },
          { $inc: { stock: line.quantity } }, { session });
        await StockLog.create([{ product: line.product, change: line.quantity,
          refType: 'purchases', refId: po._id, user: userId }], { session });
      }
      po.status = 'received';
```

```
      await po.save({ session });
    });
  } finally { session.endSession(); }
}
```

5.2.4 Cancellation and Stock Restoration

Cancelling a confirmed sales order is the reverse operation. The stock of each line item is increased again and a stock-log entry records the restoration, so the audit trail always explains the current quantity. Only orders in the confirmed state can be cancelled, which prevents restoring stock twice.

```
// server/services/salesService.js
async function cancelConfirmedOrder(orderId, userId) {
  const session = await mongoose.startSession();
  try {
    await session.withTransaction(async () => {
      const order = await SalesOrder.findById(orderId).session(session);
      if (order.status !== 'confirmed') throw new AppError('Only confirmed orders', 409);
      for (const line of order.items) {
        await Product.updateOne({ _id: line.product },
          { $inc: { stock: line.quantity } }, { session });
        await StockLog.create([{ product: line.product, change: line.quantity,
          refType: 'sales-cancel', refId: order._id, user: userId }], { session });
      }
      order.status = 'cancelled';
      await order.save({ session });
    });
  } finally { session.endSession(); }
}
```

5.2.5 Security Middleware Stack

The Express application applies the security middleware before any route. Helmet sets standard security headers, CORS accepts only the origin given in CLIENT_URL, the rate limiters slow down brute-force attempts on authentication and abuse of data routes, and express-mongo-sanitize removes keys that start with a dollar sign from incoming data so that a request such as {"$gt": ""} cannot be used for NoSQL injection.

```
// server/app.js (excerpt)
app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json({ limit: '1mb' }));
app.use(mongoSanitize());
app.use('/api/auth', authLimiter);   // strict limit on login attempts
app.use('/api', apiLimiter);         // general limit on data routes
```

5.2.6 Dashboard Aggregation Pipeline

The sales trend of the dashboard is computed in the database with an aggregation pipeline. Confirmed orders of the selected period are matched, grouped by day and sorted, and only the small result set is sent to the client, where Recharts draws the line chart.

```
// server/services/reportService.js
function salesTrend(from, to) {
  return SalesOrder.aggregate([
    { $match: { status: 'confirmed', createdAt: { $gte: from, $lte: to } } },
    { $group: {
        _id: { $dateToString: { format: '%Y-%m-%d', date: '$createdAt' } },
        revenue: { $sum: '$total' }, orders: { $sum: 1 } } },
```

```
    { $sort: { _id: 1 } },
  ]);
}
```

5.3 RESULTS

The implemented prototype was run on the seed data set, and the results observed on the dashboard, the API and the test runs are summarized in Table 5.1.

Table 5.1: Key Results of the Implemented Prototype

Metric	Result	Remarks
Functional modules implemented	7	Authentication, Inventory, Sales, Purchases, HR, Dashboard, Reports
User roles supported	3	Admin, Manager and Staff with a shared permission matrix
Seeded demo products	30	Across 5 categories with realistic SKUs and prices
Seeded historical orders	20	Sales and purchase orders with various statuses
Negative stock after any operation	0	Prevented by the conditional atomic update
End-to-end integration flow	Pass	Login, product, purchase receipt, sales confirmation, reports
Security controls in place	5	JWT, Helmet, rate limiting, CORS allow-list, sanitisation
Client production build	Successful	Vite build completes without errors

5.4 TESTING AND DISCUSSION

The system was tested with functional, role-based and integration scenarios listed in Table 5.2. The complete business workflow is also automated in the Supertest suite tests/e2e.test.js, which logs in, creates a category and a product, creates and receives a purchase order and verifies that stock increases, creates and confirms a sales order and verifies that stock decreases, and finally queries the reports and the dashboard to check that the figures are consistent.

Table 5.2: Test Scenarios and Observed Results

No.	Scenario	Expected Result	Status
1	Log in as Admin with valid credentials	Token issued and the dashboard is shown	Pass
2	Send a login request containing a NoSQL operator payload	Request is sanitised and rejected	Pass
3	Log in as Staff and open Purchases or HR	Menu item hidden; direct access returns 403	Pass

No.	Scenario	Expected Result	Status
4	Create a category and a product	Product saved with a unique SKU and listed	Pass
5	Receive a purchase order	Stock increases and a stock log is written	Pass
6	Receive the same purchase order again	Rejected; stock is not increased twice	Pass
7	Confirm a sales order above available stock	Rejected with an insufficient stock error; stock unchanged	Pass
8	Confirm a valid sales order	Stock decreases and an invoice number is generated	Pass
9	Cancel a confirmed sales order	Stock is restored and the movement is logged	Pass
10	Filter a report by date and export CSV	Aggregated data shown and the CSV file downloads	Pass

The results show that combining a transactional service layer with a single permission matrix is effective: stock stays correct under every operation, each role sees only what it is allowed to use, and the audit trail explains every change of quantity. Because the same services are used by the API, the seed script and the tests, the behaviour demonstrated in the interface is the behaviour that is verified automatically. Compared with spreadsheets and heavy enterprise suites in Table 2.1, SmartERP offers the consistency of an ERP with the simplicity and deployability of a modern web application.

The main limitations of the current version are that it handles a single warehouse and a single currency, does not integrate payment gateways or send e-mail notifications, and relies on a MongoDB replica set for transactions. These points are addressed in the future enhancements.

CHAPTER 6
CONCLUSION & FUTURE WORK

This chapter deals with the conclusion of the project and possible future enhancements.

6.1 CONCLUSION

SmartERP demonstrates that the core processes of a small business can be integrated in one reliable, secure and easy-to-use web application. The main accomplishments are:

- **Integrated Modules:** Authentication, inventory, sales, purchases, HR, dashboard and reports work on one database and one data model.
- **Transaction-Safe Stock:** MongoDB transactions guarantee that stock never becomes negative, a purchase is received only once and every movement is logged.
- **Granular Access Control:** A single shared permission matrix gives Admin, Manager and Staff exactly the access they need on both server and client.
- **Actionable Analytics:** Aggregation pipelines and Recharts present sales trends, top products and stock distribution, with date-filtered reports and CSV export.
- **Defence in Depth:** JWT, Helmet, rate limiting, a CORS allow-list and sanitisation protect the API against common attacks.
- **Verified Quality:** Unit, service and Supertest end-to-end tests prove the complete workflow from login to reports.
- **Modern Experience and Deployment:** A responsive sky-blue interface with consistent feedback states, demo seed data, a Postman collection and a deployment guide for Atlas, Render and Vercel.

6.2 FUTURE ENHANCEMENTS

1. **Cloud Deployment and Monitoring:** Complete the final roadmap phase by deploying the server on Render and the client on Vercel with a managed Atlas cluster, and add uptime and error monitoring.
2. **Demand Forecasting:** Apply time-series and machine-learning models to the sales history to forecast demand and suggest reorder quantities automatically.
3. **Multi-Warehouse and Multi-Company Support:** Track stock per location with transfers between warehouses and support several companies in one installation.

4. **Invoicing, Tax and Payments:** Generate PDF invoices with tax details and integrate online payment gateways.
5. **Notifications and Barcode Support:** Send e-mail or push alerts for low stock and order events, and support barcode scanning for fast billing and stock entry.
6. **Advanced Security and Audit:** Add refresh-token rotation, two-factor authentication and an audit-log screen for administrators.

APPENDIX

PROJECT DETAILS AND SETUP

The complete source code of SmartERP is maintained in the project repository (client, server and shared folders). This appendix records the project structure, the roles available for demonstration, the steps to run the system locally and the deployment settings.

1. Project Structure

```
smarterp/
├── client/                # React.js + Vite + Tailwind CSS front end (port 5173)
├── server/                # Node.js + Express.js REST API and service layer (port 5000)
│   ├── app.js             # Express app: security middleware and routes
│   ├── server.js          # Entry point of the backend
│   ├── scripts/seed.js    # Demo data seeder
│   └── tests/             # Jest and Supertest suites, including e2e.test.js
├── shared/
│   └── permissions.json   # Single source of truth for role permissions
├── docs/FINAL_REPORT.md   # Deployment guide, test report and demo script
├── SmartERP_Postman_Collection.json
└── README.md              # Overview, roles, setup and API summary
```

2. Demo Roles

Demo Roles and Permissions

Role	Demo Account	Special Permissions
Admin	admin@smarterp.com	Full access to all modules, user and role management and system settings
Manager	manager@smarterp.com	Inventory CRUD, sales and purchases, reports, view employees; no user management
Staff	staff@smarterp.com	View inventory, create sales orders, limited dashboard with cost and profit hidden

The default demo passwords are listed in the project README and must be changed before any real deployment.

3. How to Run Locally

```
# Install dependencies and load the demo data (from the server folder)
cd server
npm install
npm run seed


# Start the backend and the frontend together (from the project root)
cd ..
npm run dev
```

The client is served at http://localhost:5173 and the API at http://localhost:5000 as configured for the live demonstration; ports can be changed in the project configuration.

4. Environment Variables (server/.env)

```
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173
MONGO_URI=mongodb+srv://<user>:<password>@<cluster>.mongodb.net/smarterp
JWT_SECRET=<long-random-secret>
JWT_EXPIRE=30d
ADMIN_EMAIL=<admin-email>
ADMIN_PASSWORD=<strong-password>

# client/.env
VITE_API_URL=http://localhost:5000/api
```

5. Deployment Settings

```
# Render (Web Service for the API)
Root Directory : server
Build Command  : npm install
Start Command  : npm start
Environment    : PORT, NODE_ENV=production, CLIENT_URL, MONGO_URI, JWT_SECRET, JWT_EXPIRE

# Vercel (client) - environment key and vercel.json rewrite for client-side routing
Root Directory : client
Environment    : VITE_API_URL=https://<render-service>.onrender.com/api
{ "rewrites": [ { "source": "/(.*)", "destination": "/index.html" } ] }
```

SCREENSHOTS OF THE PROJECT

The following figures document the running SmartERP application. Replace each placeholder with the corresponding screenshot of your system.

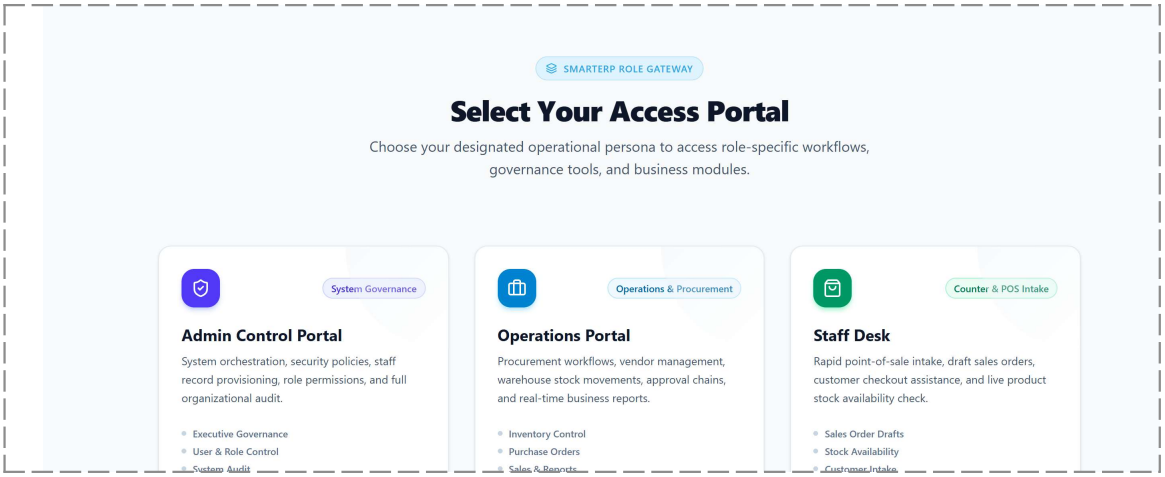


Figure A.1: Login Page

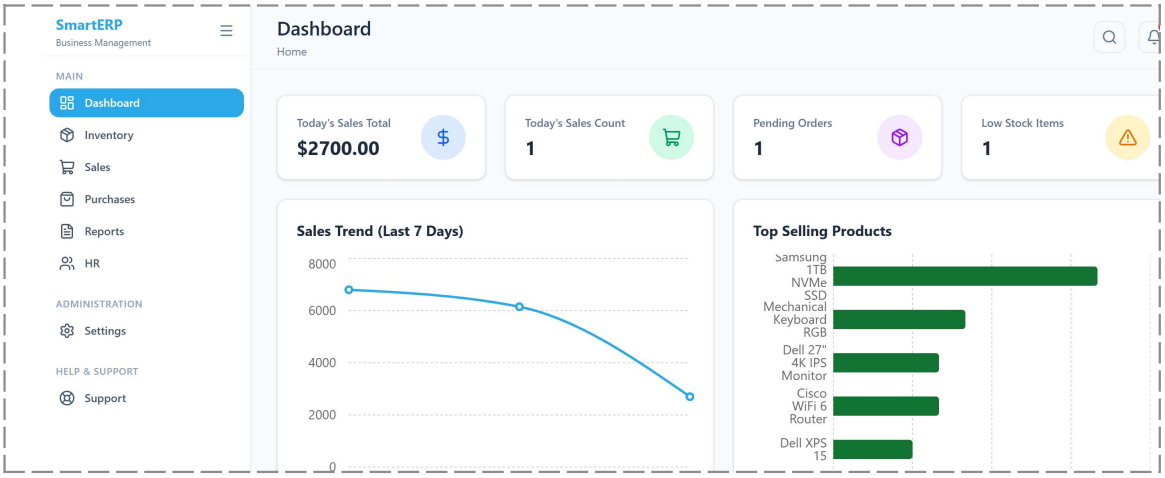


Figure A.2: Executive Dashboard

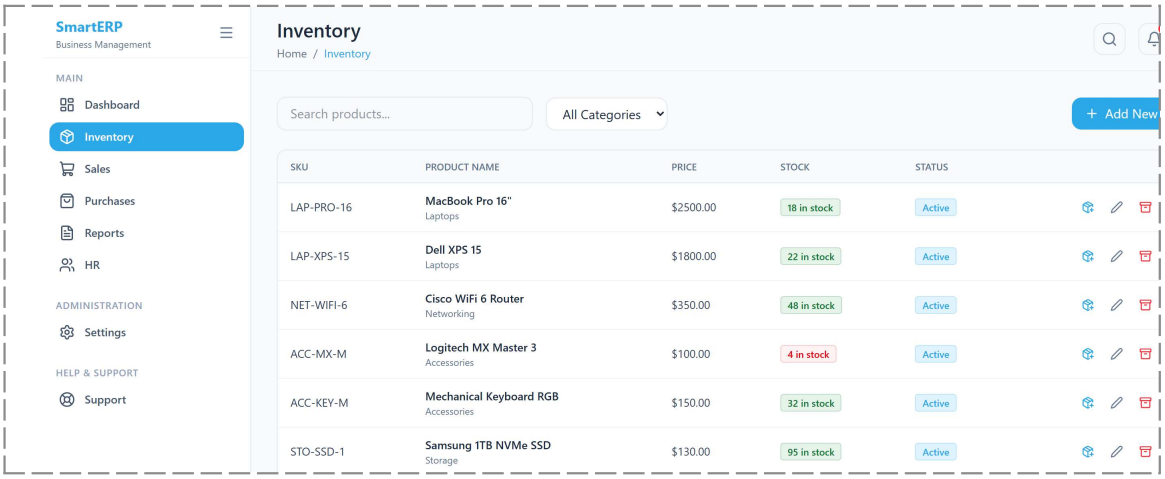


Figure A.3: Inventory Product List

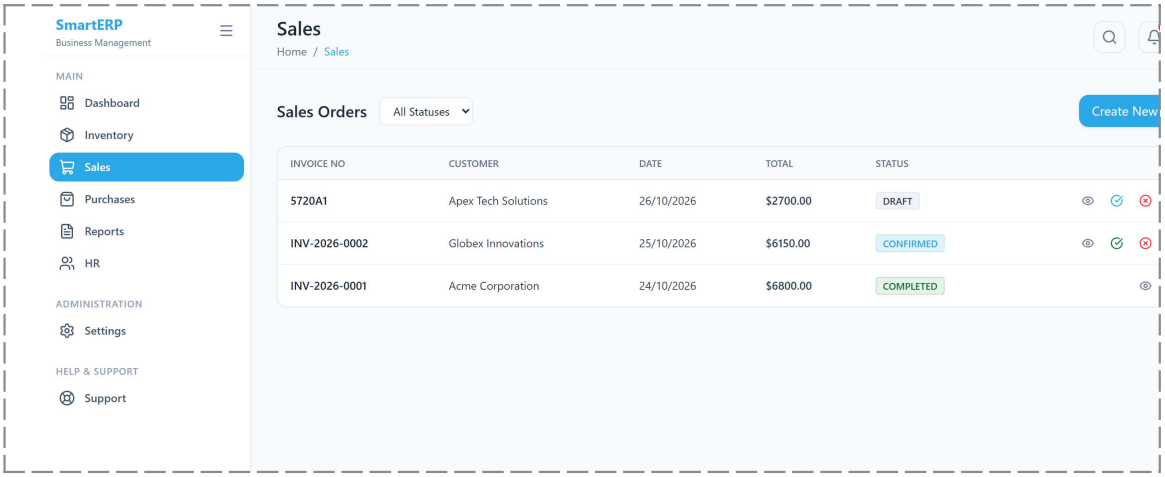


Figure A.4: Sales Order and Order Confirmation

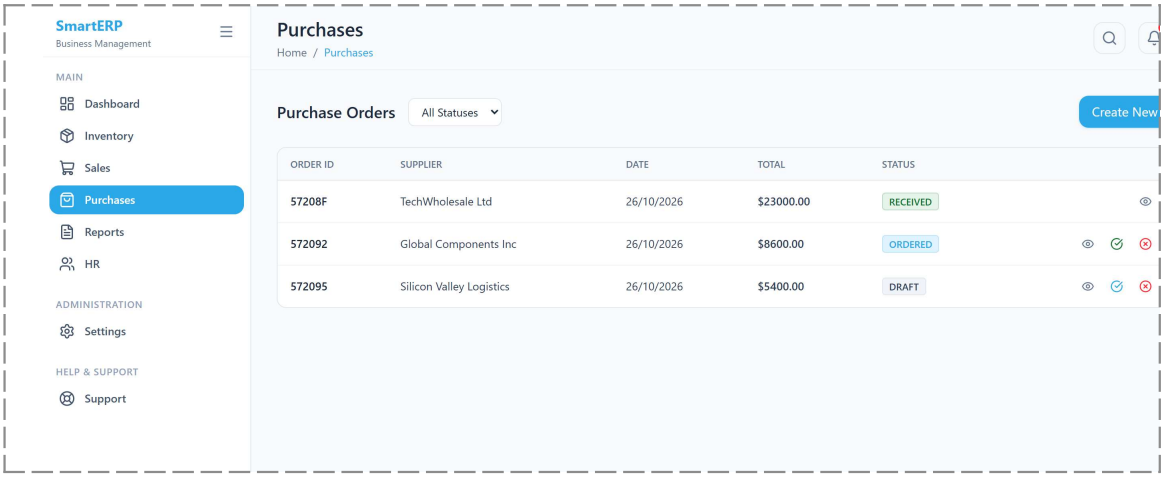


Figure A.5: Purchase Order and Receive Stock

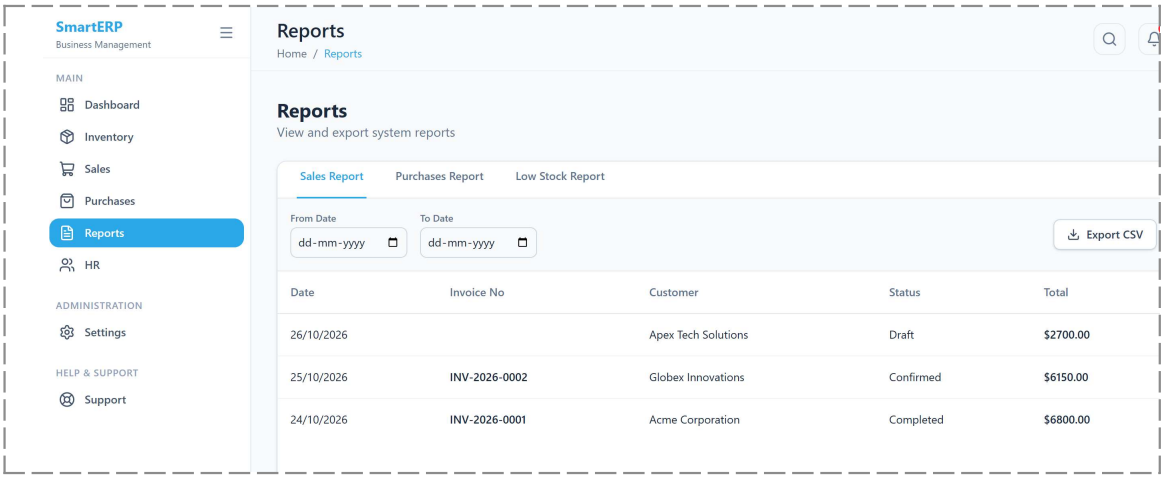


Figure A.6: Reports and CSV Export

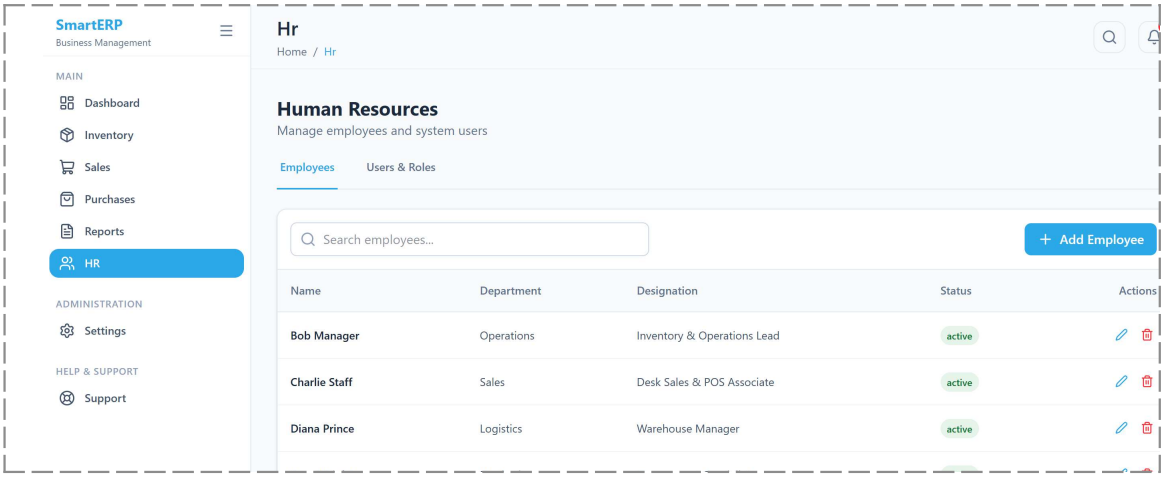


Figure A.7: Employee Directory (HR Module)

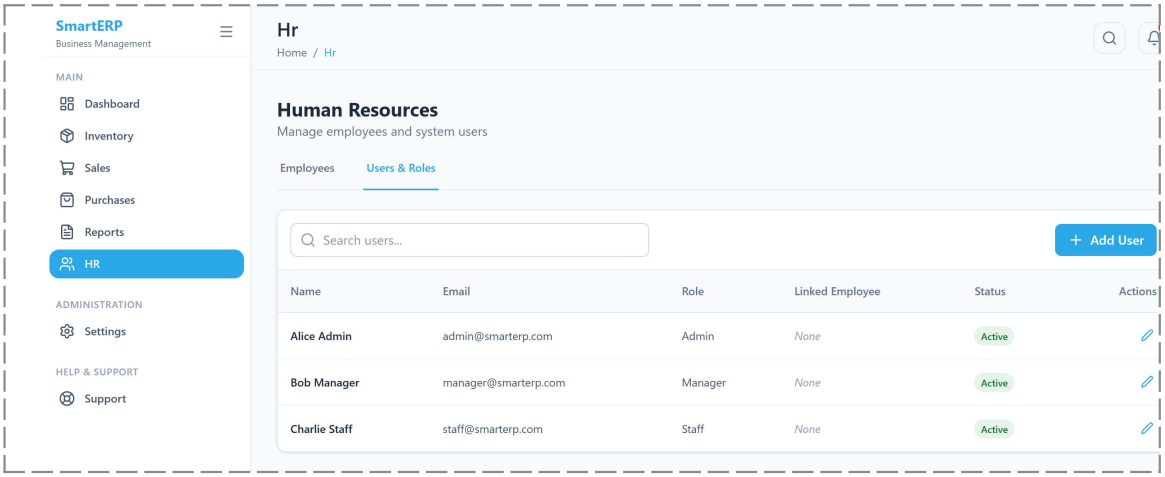


Figure A.8: Users and Roles Management

REFERENCES

[1] T. H. Davenport, "Putting the enterprise into the enterprise system," Harvard Business Review, vol. 76, no. 4, pp. 121–131, 1998.

[2] E. F. Monk and B. J. Wagner, "Concepts in Enterprise Resource Planning," 4th ed., Cengage Learning, Boston, MA, USA, 2013.

[3] T. Härder and A. Reuter, "Principles of transaction-oriented database recovery," ACM Computing Surveys, vol. 15, no. 4, pp. 287–317, 1983.

[4] J. Gray and A. Reuter, "Transaction Processing: Concepts and Techniques," Morgan Kaufmann, San Francisco, CA, USA, 1993.

[5] D. Ferraiolo and R. Kuhn, "Role-based access controls," in Proc. 15th NIST-NCSC National Computer Security Conference, Baltimore, MD, USA, 1992, pp. 554–563.

[6] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[7] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[8] OWASP Foundation, "OWASP Top Ten 2021," [Online]. Available: https://owasp.org/Top10/

[9] M. Kleppmann, "Designing Data-Intensive Applications," O'Reilly Media, Sebastopol, CA, USA, 2017.

[10] I. Sommerville, "Software Engineering," 10th ed., Pearson Education, London, UK, 2015.

[11] MongoDB Inc., "MongoDB Manual: Transactions," MongoDB Documentation. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[12] Express.js, "Express - Node.js web application framework," [Online]. Available: https://expressjs.com

[13] React Team, "React Documentation," [Online]. Available: https://react.dev

[14] Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: https://tailwindcss.com/docs

[15] Recharts, "Recharts: A composable charting library built on React components," [Online]. Available: https://recharts.org

[16] Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/docs